CHAPTER 1 Introduction to PL/SQL

1.1 What Is PL/SQL?

A computer *language* is a particular way of giving instructions to (that is, programming) a computer. Computer languages tend to have a small vocabulary compared to regular human language. In addition, the way you can use the language vocabulary—that is, the grammar—is much less flexible than human language. These limitations occur because computers take everything literally; they have no way of reading between the lines and assuming what you Intended.

Procedural refers to a series of ordered steps that the computer should follow to produce a result. This type of language also includes data structures that hold information that can be used multiple times. The individual statements could be expressed as a flow chart (although flow charts are out of fashion these days). Programs written in such a language use its sequential, conditional, and Iterative constructs to express *algorithms*. So this part of the PL/SQL's definition is just saying that it is in the same family of languages as BASIC, COBOL, FORTRAN, Pascal, and C.

Language Categories

Saying that PL/SQL is a procedural language makes more sense when you understand some other types of programming languages. There are at least four ways to categorize popular languages.

Procedural programming languages

Allow the programmer to define an ordered series of steps to follow in order to produce a result. Examples: PL/SQL, C, Visual Basic, Perl, Ada.

Object-oriented programming languages

Based on the concept of an *object*, which is a data structure encapsulated with a set of routines, called *methods* that operate on the data. Examples: Java, C++, JavaScript, and sometimes Perl and Ada 95.

Declarative programming languages

Allow the programmer to describe relationships between variables in terms of functions or rules; the language executor (interpreter or compiler) applies some fixed algorithm to these relations to produce
a result. Examples: Logo, LISP, and Prolog.

Markup languages

Define how to add information into a document that indicates its logical components or that provides layout instructions. Examples:
HTML, XML.

Structured Query Language is a language based on set theory, so it is all about manipulating sets of data. SQL consists of a relatively small number of main commands such as SELECT, INSERT, CREATE, and GRANT; in fact, each statement accomplishes what might take hundreds of lines of procedural code to accomplish. That's one reason SQL-based databases are so widely used.

Learning Oracle PL/SQL

PL/SQL, Oracle's programming language for stored procedures, delivers a world of possibilities for your database programs. PL/SQL supplements the standard relational database language, SQL, with a wide range of procedural features, including loops, IF-THEN statements, advanced data structures, and rich transactional control--all closely integrated with the Oracle database server.

PL/SQL is a procedural structured Query Language, is an extension to SQL where we can write programs using all the SQL statements and procedural statements.

Various procedural statements we can use in PL/SQL are

- Assignment statements
- Conditional statements
- Loops
- Transactional processing statements

- Assignment statements In any programming language, we use = as assignment operator and == as comparison operator. Where as in PL/SQL we use = as comparison operator and: = as assignment operator and the statement used with this operator is called as assignment statement.
  Ex. c: = a + b;

- Conditional statements Generally in any programming language, we use If statement as conditional statement.

Syntax: - Simple IF condition

```
IF <CONDITION>  THEN
    ST1;
    ST2;
ELSE
    ST3;
END IF;
```

Nested IF

```
IF <CONDITION1> THEN
    ST1;
    ST2;
ELSIF <CONDITION2> THEN
    ST3;
    ST4;
ELSIF <CONDITION3> THEN
    ST5;
ELSE
    ST6;
END IF;
```

In addition to these IF statements, we can also use SQL functions DECODE () and CASE.

- Loops In any programming language, we use tow different types of Loops
    1. Iterative Loops
    2. Conditional Loops

1. Iterative Loops These loops performs operation for a range of values. For loop is used as iterative loop.

```
FOR loop_counter IN [REVERSE] lower_bound .. upper_bound
LOOP
Statements
END LOOP;
```

Where:

loop_counter

An identifier that has not been declared in the program, this variable gives you a way of detecting the “trip number” through the loop.

Lower_bound

A numeric expression that Oracle uses to compute the smallest value assigned to loop_*counter*. Often, this will just be the number 1. You should make this an integer, but if you don’t, PL/SQL automatically rounds it to an integer. If the lower bound is greater than the upper bound, the loop will not execute; if it is null, your program will end in a runtime error.

REVERSE

Without this keyword, the loop counter increases by one with every trip through the loop, from the lower to the upper bound. With REVERSE, though, the loop will *decrease* by one instead, going from the upper to the lower bound.

```
FOR n IN REVERSE 1..3
LOOP
…
END LOOP;
```

Also, the low and high values in the FOR loop range do not have to be literals, as you can see in the next example:

```
FOR month_num IN 1 .. TO_NUMBER(TO_CHAR(SYSDATE, 'MM'))
LOOP
Statements;
END LOOP;
```

As you're working with loops, it's important to know that PL/SQL declares the loop counter variable for you automatically, and there can be problems if it has the same name as a variable you've declared in the usual place (the program's declaration section). The scope (the only part of the code in which it can be referenced) of the loop counter is between the LOOP and END LOOP keywords.

FOR Loops

- Use a FOR loop to shortcut the test for the number of iterations
- Do not declare the counter; it is declared implicitly
- Lower_bound **..** upper_bound is required
- Reference the counter within the loop only; it is undefined outside the loop.
- Do not reference the counter as the target of an assignment.

Conditional Loops

The simple loop is the, well, simplest loop structure. It has the following syntax:

```
LOOP
EXIT WHEN <CONDITION>;
Statements
END LOOP;
```

We can use this exit condition any where within the loop.

While Loop

```
While <Condition>
LOOP
   Statements;
End Loop;
```

Transactional Processing Statements

We can use COOMIT and ROLLBACK as transactional processing statements. Once the transaction is over then you decide whether you save the data or not to save the data.

Note: - PL/SQL supports 4GL (object oriented programming language) features.

Every language uses some structure in writing programs. PL/SQL also uses two different types of block structures in writing programs.

1. Unnamed PL/SQL block structure
2. Named PL/SQL block structure

1. **Unnamed PL/SQL block structure**

They are also called as anonymous blocks; they do not have names in the database.

Structure of anonymous PL/SQL block structure

```
DECLARE
        Declaration section  (optional)
BEGIN
        Executable section  (mandatory)
EXCEPTION
        Exception section  (optional)
END;
```

- Between DECLARE and BEGIN we can declare all the variables, constants which are going to be used in the program. It is called as DECLARATIONSECTION (optional).

- The BEGIN and END keywords are mandatory and enclose the body of actions to be performed. This section is referred to as the EXECUTABLE SECTION (mandatory) where we write actual logic of the program.

- The section between EXCEPTION and END is referred to as the exception section. The exception section traps error conditions.

Errors are of two types.

a) Primitive errors
b) Logical or runtime errors

Primitive (syntax) errors are handled at the time of compiling the program itself.

Logical or runtime errors are handled only at runtime.

These logical errors are handled in the EXCEPTION SECTION.

This section is optional.

The keywords DECLARE, semicolons do not follow BEGIN and EXCEPTION, but END and all other PL/SQL statements do require statement terminators.

Summary

Section	Description	Inclusion
Declarative	Contains all variables, constants, cursors, and user-defined exceptions that are referenced in the executable and declarative sections	Optional
Executable	Contains SQL statements in manipulate data in the database and PL/SQL statements to manipulate data in the block	Mandatory
Exception	Specifies the sections to perform when errors and abnormal conditions arise in the executable section	Optional

Naming Rules for Identifiers

Identifiers are the names given to PL/SQL elements such as variables, procedures, variables, and user-defined types. These names must follow certain rules of the road; namely, they:

- Are no more than 30 characters in length. Start with a letter.
- Consist of any of the following: letters, numerals, the dollar sign ($), the hash sign (#), and the underscore (_).
- Cannot be the same as a PL/SQL reserved word.
- Are unique within its scope. You cannot, in other words, declare two variables with the same name in the same block of code.

These are valid names for variables:

```
Birthdate
vote_total
sales_year7
contribution$
item#
```

These names, on the other hand, will cause compilation errors:

```
the_date_of_birth_of_my_grandchildren -- TOO LONG
1st_choice -- STARTS WITH DIGIT
Myemail@stevenfeuerstein.com -- CONTAINS INVALID CHARACTER
```

Scope of Variables

The *scope* of a variable is the portion of PL/SQL code in which that variable can be referenced (i.e., its value read or modified). Most of the variables you define will have as their scope the block in which they were defined. Consider the following block:

```
DECLARE
book_title VARCHAR2 (100);
BEGIN
book_title := 'Learning Oracle PL/SQL';
END;
/
```

The variable `book_title` can be referenced within this block of code, but nowhere else. So if I happen to write another separate block of code, any attempt to read or change the value of `book_title` will result in a compilation error:

```
SQL> BEGIN
2 IF book_title LIKE '%PL/SQL%'
3 THEN
4 buy_it;
5 END IF;
6 END;
```

Types of Variables

- PL/SQL variables:

All PL/SQL variables have a data type, which specifies a storage format, constants, and valid range of values. PI/SQL supports four data type categories---

- Scalar
- Composite
- Reference
- LOB(large objects)

- Non-PL/SQL variables:

They include host language variables declared in precompiler programs, screen fields in Forms applications, and SQL*Plus or iSQL*Plus host variables.

Using SQL or iSQL*plus Variables Within PL/SQL Blocks

- PL/SQL does not have input or output capability of its own.
- You can reference substitution variables within a PL/SQL block with a preceding ampersand.
- ISQL*Plus host(or bind) variables can be used to pass run time values out of the PL/SQL block back to the iSQL*Plus environment.

Declaration section

A variable in PL/SQL can be declared in two different ways

1) Declaring variable by giving its data type and size

Syntax :

Identifier [CONSTANT] datatype [NOT NULL] [:= | DEFAULT expr];

Example

```
V_joindate   DATE;
V_empno      Number(3) NOT NULL     := 1001;
V_location   VARCHAR2(10)           := 'sec-bad';
C_comm.      CONSTANT NUMBER:       = 1400;
```

Note: -

1. **=** Used as comparison operator and **:=** as assignment operator.
2. Naming convention for variable, variable name starts with "v" and constant start with "c".
3. We can also give integrity constraints at the time of declaring variable Vename varchar2 (10) not null.
4. Only one variable is allowed to declare in each line.
5. Follow naming conventions.

Scalar Data Types

- Hold a single value
- Have no internal components

Base Scalar Data Types

Data Type	Description
CHAR [(max-Len)]	Base type for fixed-length character data up to 32,767 bytes. If you do not specify a maximum length, the default length is set to 1.
VARCHAR2 (max-Len)	Base type for variable-length character data up o 32,767 bytes. There is no default size for VARCHAR2 variables and constants.
LONG	Base type for variable-length character data up to 32,760 bytes. Use the LONG data type to store variable-length character strings. You can insert any LONG value into a LONG database column because the maximum width of a LONG column is 2 ** 31 bytes. However, you cannot retrieve a value longer than 32760 bytes from a LONG column into a LONG variable.
Data type	
LONG RAW	Base type for binary data and byte strings up to 32,760 bytes.
NUMBER(precision, scale)	Number having precision p and scale s. the precision p can range from 1 to 38.
BINARY_INTEGER	Base type for integers between –2,147,483,647 and 2,147,483,647
PLS_INTEGER	BASE type for signed integers between –2,147,483,647 and 2,147,483,647. PLS_INTEGER values require less storage and are faster than NUMBER and BINARY_INTEGER values.
BOOLEAN	Base type that stores one of three possible values
DATE	Base type for date and time

The %TYPE Attribute

A variable declared by referring column of a table. We can also use %TYPE attribute to declare a variable according to another previously declared variable.

Example

1. V_empno EMP.empno%type;
2. v_salary NUMBER(8,2);
3. v_comm. V_salary%type;

Note :- A NOT NULL database column constraint does not apply to variables that are declared using %TYPE. Therefore, if you declare a variable using the

%TYPE attribute that uses a database column defined as NOT NULL, you can assign the NULL value to the variable.

Composite Data Types

A scalar type has no internal components. A composite type has internal components that can be manipulated individually. Composite data types (Also known as collections) are of TABLE,RECORD, NESTED

<variable> <Tablename>%rowtype;

rec emp%rowtype;

Use of variables

Variables can be used for:

- Temporary storage of data Data can be temporarily stored in one or more variables for use when validating data input and for processing later in the data flow process.
- Manipulation of stored values Variables can be used for calculation and other data manipulations without accessing the database.
- Reusability After they declared, they can be used repeatedly in an application simply by referencing them in other statements, including other statements, including other declaratie statements.
- Ease of maintenance When we declare variables using %TYPE and %ROWTYPE, if any underlying definition changes, the variable definition changes accordingly at run time. ***This provides data independence, reduces maintenance costs.***

Program

Write a program to maintain students information with
Rollno, Student_name, sub1,sub2, Total,average
Where sub1,sub2 are the marks in different subjects.

Solution

Step 1 create student table with above structure
Step 2 In SQL*plus environment, open the system editor using a command
SQL> EDIT student
Or

In iSQL*plus environment, write the code in Executable window and save the script into a file.

```
DECLARE
        Stu         STUDENT%rowtype;
   BEGIN
    -- generate rollno automatically

SELECT nvl(max(rollno),0) + 1 into stu.rollno from student;

-- input name and marks in two subjects

stu.sname  := '&name';
stu.sub1    := &marks1;
stu.sub2    := &marks2;
Stu.total     := stu.sub1 + stu.sub2;
Stu.average:= stu.total/2;
INSERT INTO STUDENT VALUES stu;
COMMIT;
END;
/
```

To run PL/SQL program

From SQL*Plus Environment

SQL> start student
Or
SQL> @student

From iSQL*Plus Environment

Use **Execute** button

Bind Variables

A bind variable is a variable that you declare in a host environment. Bind variables can be used to pass run-time values, either number or character, into or out of one or more PL/SQL programs.

The PL/SQL program use bind variables as they would use any other variable. You can reference variables declared in the host or calling environment in PL/SQL statements, unless the statement is in a procedure, function, or package.

Creating Bind Variables

To declare a bind variable In the iSQL*Plus environment, use the command **VARIABLE**.

For example, we can declare a variable of type number and VARCHAR2 as follows:

```
VARIABLE vsal NUMBER
VARIABLE getmsg as VARCHAR2 (30)
```

Both SQL and iSQL*Plus environment can reference the bind variable, and iSQL*Plus can display its value through PRINT command

Using Bind Variables

To reference a bind variable in PL/SQL, you must prefix its name with a colon (**:**)

Example

```
SQL>VARIABLE g_salary number

BEGIN
   SELECT sal INTO :g_salary FROM EMP WHERE empno = 7521;
END;
/

SQL> PRINT g_salary
```

Referencing Non-PL/SQL Variables

To reference host-variables, you must prefix the references with a colon (**:**) to distinguish them from declared PL/SQL variables

Example

```
VARIABLE g_month_sal NUMBER
DEFINE     p_annual_sal = 50000
SET VERIFY OFF
DECLARE
     V_sal   NUMBER(9,2) := &p_annual_sal;   -- reads the value from host variable
 BEGIN
    :g_monthly_sal := v_sal/12;
END;
/
PRINT  g_monthly_sal
```

Note The DEFINE command specifies a user variable and assigns it a CHAR value. Even though you enter the number 50000, iSQL*Plus assigns a CHAR value to p_annual_sal consisting of the characters, 5,0,0,0 and 0.

Programming Guidelines

Make code maintenance easier by:

- Documenting code with comments
- Developing a case convention for the code
- Developing naming conventions for identifiers and other objects
- Enhancing readability by Indenting

Code Conventions

The following table provides guidelines for writing code in uppercase or lowercase to help you distinguish keywords from named objects.

Category	Case Convention	Examples
SQL statements	Uppercase	SELECT, INSERT
PL/SQL keywords	Uppercase	DECLARE,BEGIN,IF
Data types	Uppercase	VARCHAR2, BOOLEAN
Identifiers and parameters	Lowercase	V_sal_emp_cursor, g_sal
Database tables and	Lowercase	Emp, empno

columns		

Summary

A Pl/SQL block is a basic, unnamed unit of a Pl/SQL program. It consists of a set of SQL or PL/SQL statements and it performs a single logical function. The declarative part is the first part of a PL/SQL block and is used for declaring objects such as variables, constants, cursors and exceptions. The executable part is the mandatory part of a PL/SQL block, and contains SQL and PL/SQL statements for querying and manipulating data. The exception-handling part is embedded inside the executable part of a block and is placed at the end of the executable part.

An anonymous PL/SQL block is the basic, unnamed unit of a PL/SQL program. Procedures and functions can be compiled separately and stored permanently in an Oracle database, ready to be executed.

2.0 Exception Handling

Objectives
After completing this lesion, you should be able to do the following:
▪ **Define PL/SQL exceptions** ▪ **Recognize unhandled exceptions** ▪ **List and use different types of PL/SQL exception handlers** ▪ **Trap unanticipated errors** ▪ **Describe the effect of exception propagation in nested blocks** ▪ **Customize PL/SQL exception messages**

2.1 Introduction

In PL/SQL, errors and warnings are called as exceptions. Whenever a predefined error occurs in the program, PL/SQL raises an exception. For example, if you try to divide a number by zero then PL/SQL raises an exception called ZERO_DIVIDE and if SELECT can not find a record then PL/SQL raises exception No_DATA_FOUND.

PL/SQL has a collection of predefined exceptions. Each exception has a name. These exceptions are automatically raised by PL/SQL whenever the corresponding error occurs.

In addition to predefined exceptions, user can also create his own exceptions to deal with errors in the application.

An exception is an identifier in PL/SQL that is raised during the execution of a block that terminates its main body of actions. A block always terminates when PL/SQL raises an exception, but can you specify an exception handler to perform final actions.

PL/SQL allows developers to raise and handle errors (exceptions) in a very flexible and powerful way. Each PL/SQL block can have its own exception section, in which exceptions can be trapped and handled (resolved or passed on to the enclosing block).

When an exception occurs (is raised) in a PL/SQL block, its execution section immediately terminates. Control is passed to the exception section.

Every exception in PL/SQL has an error number and error message; some exceptions also have names.

Handling Exceptions with PL/SQL

An exception is an identifier in PL/SQL that is raised during execution.

How is it raised?

- **An Oracle error occurs.**
- **You raise it explicitly.**

How do you handle it?

- **Trap it with a handler.**
- **Propagate it to the calling environment.**

2.1.1 Declaring Exceptions

Some exceptions (see the following table) have been pre-defined by Oracle in the STANDARD package.
You can also declare your own exceptions as follows:

```
DECLARE
exception_name EXCEPTION;
```

An exception can be declared only once in a block, but nested blocks can declare an exception with the same name as an outer block. If this multiple declaration occurs, scope takes precedence over name when handling the exception. The inner block's declaration takes precedence over a global declaration.

When you declare your own exception, you must RAISE it explicitly. All declared exceptions have an error code of 1 and the error message "User-defined exception," unless you use the EXCEPTION_INIT pragma.

You can associate an error number with a declared exception with the PRAGMA EXCEPTION_INIT statement:

```
DECLARE
Exception_name EXCEPTION;
PRAGMA EXCEPTION_INIT (exception_name,error_number);
```

Where *error_number* is a literal value (variable references are not allowed). This number can be an Oracle error, such as -1855, or an error in the user-definable -20000 to -20999 range.

2.1.2 Raising Exceptions

An exception can be raised in three ways:
·

- By the PL/SQL runtime engine
- · By an explicit RAISE statement in your code
- · By a call to the built-in function RAISE_APPLICATION_ERROR

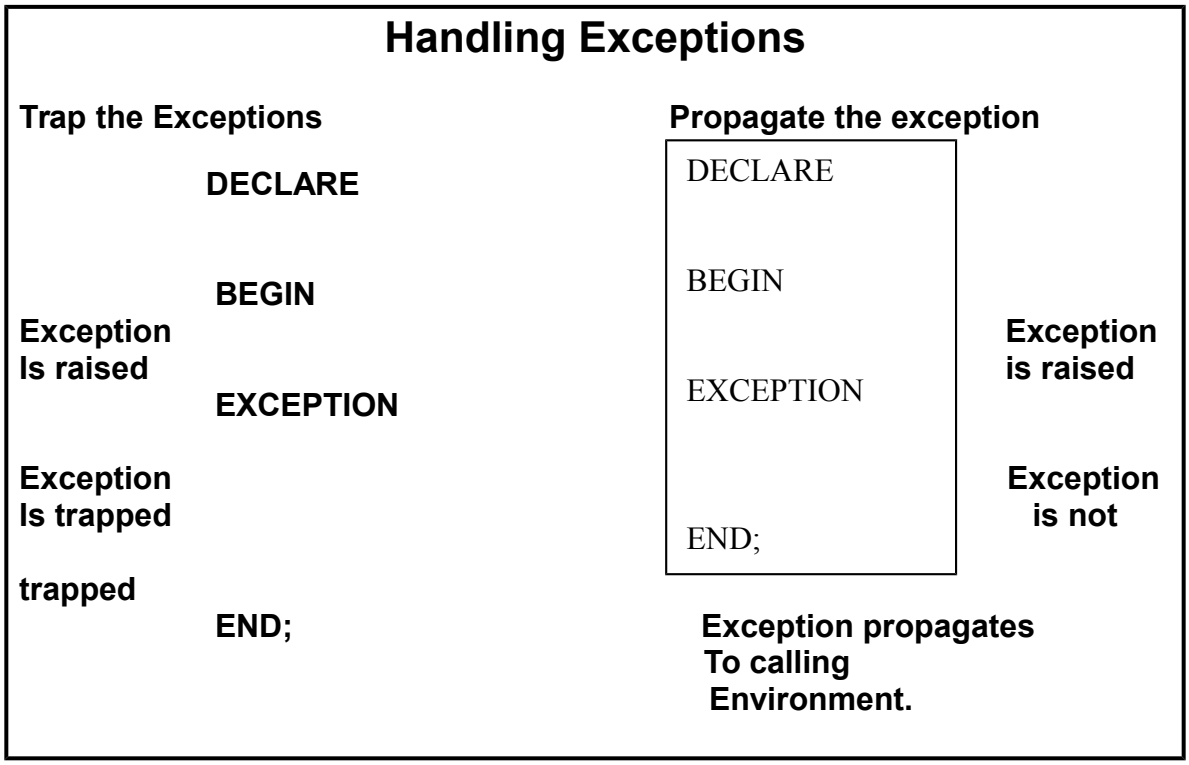
The syntax for the RAISE statement is:

```
RAISE exception_name;
```

Where *exception_name* is the name of an exception that you have declared, or that is declared in the STANDARD package.

If you use the RAISE statement inside an exception handler, you can leave off an exception name to re-raise

Handling Exceptions



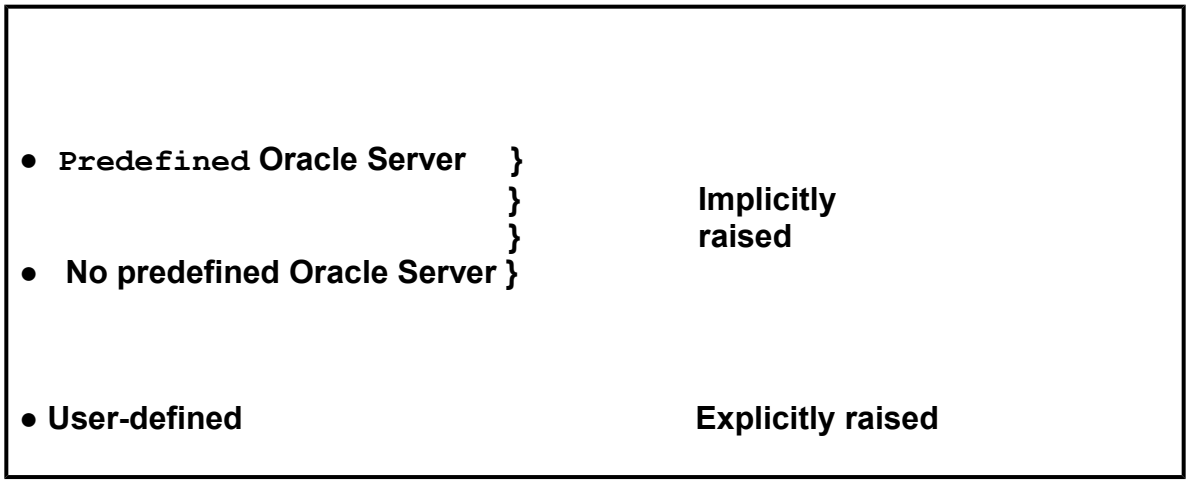
Trapping an Exception

If the exception is raised in the executable section of the block, processing branches to the corresponding exception handler in the exception section of the block. If PL/SQL successfully handles the exception, then the exception does not propagates to the enclosing block or environment. The PL/SQL block terminates successfully.

Propagating an Exception

If the exception is raised in the executable section of the block and there is no corresponding exception handler, the PL/SQL block terminates with failure and the exception is propagated to the calling environment.

Exception Types



You can program for exceptions to avoid disruption at run time. There are three types of exceptions.

Exception	Description	Directions for Handling
Predefined Oracle Server error	One of approximately 20 errors that occur most often in PL/SQL code	Do not declare and allow the Oracle server to raise them implicitly
No predefined Oracle Server error	Any other standard Oracle Server error	Declare within the declarative section and allow the Oracle Server to raise them implicitly.
User-defined error	A condition that the developer determines is abnormal	Declare within the declarative section, and raise explicitly.

Trapping Exceptions

Syntax

```
EXCEPTION
     WHEN   exception1  [  OR  exception2  … ] THEN
          Statement1;
          Statement2;

      WHEN exception3   [   OR  exception4 …]   THEN
           Statement3;
           Statement4;
       WHEN OTHERS THEN
            Statement5;
            Statement6;
```

The following PL/SQL block attempts to select information from the employee and includes an exception handler for the case in which no data is found:

```
DECLARE
vempno NUMBER;
BEGIN
SELECT empno INTO vempno FROM EMP WHERE ename = 'RAM';
EXCEPTION
WHEN NO_DATA_FOUND THEN
INSERT INTO emp (empno, ename, job, deptno)
VALUES
(101,'RAM', 'EXECUTIVE', 10);
END;
```

In other words, if I am not already an employee in the company, the SELECT statement fails and control is transferred to the exception section (which starts with the keyword EXCEPTION). PL/SQL matches up the exception raised with the exception in the WHEN clause (NO_DATA_FOUND is a named, internal

exception that represents ORA-01403-no data found). It then executes the statements in that exception handler, so I am promptly inserted into the employee table.

The WHEN OTHERS clause

EXCEPTION
WHEN OTHERS THEN
...
Use the WHEN OTHERS clause in the exception handler as a catch all to trap any exceptions that are not handled by specific WHEN clauses in the exception section. If present, this clause must be the last exception handler in the exception section.

The OTHERS handler traps all exceptions not already trapped. Some Oracle tools have their own predefined exceptions that you can raise to cause events in the application. The OTHERS handler also traps these exceptions.

Trapping Exceptions Guidelines

- The EXCEPTION keyword starts exception-handling section.
- several exception handlers are allowed.
- only one handler is processed before leaving the block.
- WHEN OTHERS is the last clause.
- Exceptions cannot appear in assignment statements or SQL statements.
- You can have only one OTHERS clause.

Trapping Predefined Oracle Server Errors

■ Reference the standard name in the Exception-handling routine.

■ Sample predefined exceptions:

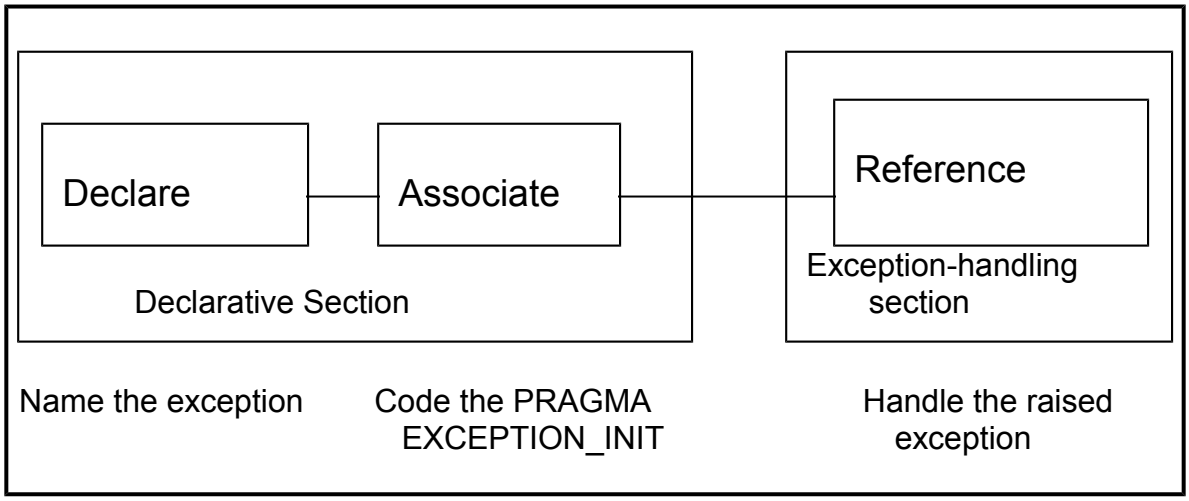
- NO_DATA_FOUND
- TOO_MANY_ROWS
- INVALID_CURSOR
- ZERO_DIVIDE
- DUP_VAL_ON_INDEX

For a complete list of predefined exceptions, see PL/SQL User's Guide and Reference "Error Handling"

Note : PL/SQL declares predefined exceptions in the STANDARD package.

It is good idea to always handle the NO_DATA_FOUND and TOO_MANY_ROWS exceptions, which are the most common.

Trapping Nonpredefined Oracle Server Errors

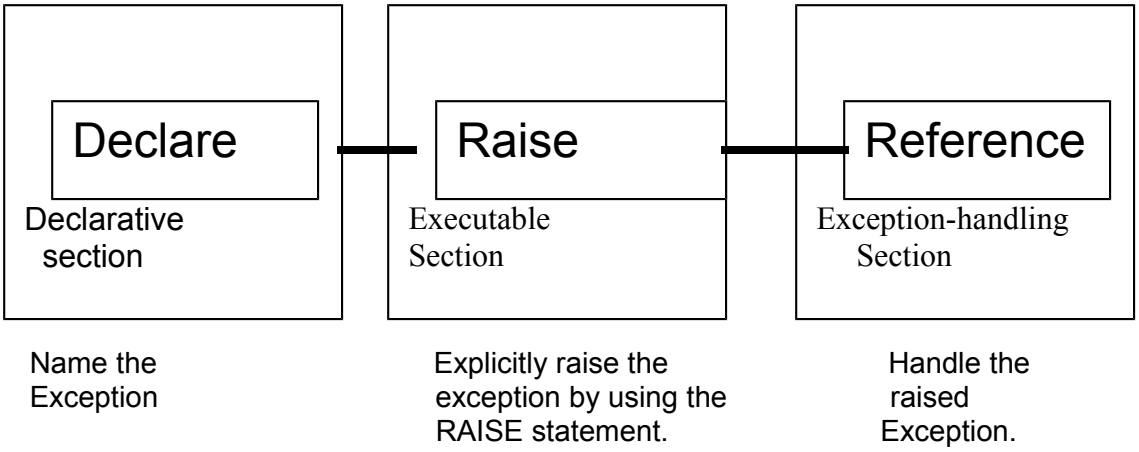



```
declare
   deptno_in number;
 still_have_employees EXCEPTION;
 PRAGMA EXCEPTION_INIT(still_have_employees,  -2292);
 BEGIN
  deptno_in := &deptno;
 DELETE FROM d1
 WHERE deptno = deptno_in;
 EXCEPTION
 WHEN still_have_employees
 THEN
 DBMS_OUTPUT.PUT_LINE
 ('Please delete employees in dept first');
 ROLLBACK;
-- RAISE; /* Re-raise the current exception. */
 END;
```

You trap a nonpredefined Oracle server error by declaring it first, or by using the OTHERS handler. The declared exception is raised implicitly. In PL/SQL, the PRAGMA_EXCEPTION_INIT tells the compiler to associate an exception name with an Oracle error number. That allows you to refer to any internal exception by name and to write a specific handler for it.

Note: PRAGMA (also called pseudoinstructions) is the keyword that signifies that the statement is a compiler directive, which is not processed when the PL/SQL block is executed. Rather, it directs the PI/SQL compiler to interpret all occurrences of the exception name within the block as the associated Oracle server error number.

Trapping User-Defined Exceptions



PL/SQL allows you to define your own exceptions. User-defined PL/SQL exceptions must be:

- Declared in the declare section of a PL/SQL block
- Raised explicitly with RAISE statements

Example:

```
DEFINE p_department_desc = 'Accounts'
DEFINE p_department_number = 50
```

```
DECLARE
  E_invalid_department  EXCEPTION;
BEGIN
   UPDATE dept
   SET                dname = '&p_department_desc'
   WHERE              deptno = &p_department_number;

    IF  SQL%NOTFOUND THEN
        RAISE e_invalid_department;
    END IF;
    COMMIT;
 EXCEPTION
      WHEN e_invalid_department THEN
        DBMS_OUTPUT.PUT_LINE('No such department id');
  END;
```

Program 1
Write a program to maintain students information with
Rollno, Student_name, sub1,sub2, Total,average
Where sub1,sub2 are the marks in different subjects.

Solution

Step 1 create student table with above structure
Step 2 In SQL*plus environment, open the system editor using a command
SQL> EDIT student

Or

In iSQL*plus environment, write the code in Executable window and save the script into a file.

```
DECLARE
        Stu         STUDENT%rowtype;
   Negative       EXCEPTION;
BEGIN
    -- generate rollno automatically

SELECT nvl(max(rollno),0) + 1 into stu.rollno from student;

-- input name and marks in two subjects

stu.sname  := '&name';
stu.sub1     := &marks1;
stu.sub2     := &marks2;
IF stu.sub1 < 0 or stu.sub2  < 0 then
   RAISE negative;
END IF;
Stu.total      := stu.sub1 + stu.sub2;
Stu.average:= stu.total/2;
INSERT INTO STUDENT VALUES stu;
COMMIT;
EXCEPTION
  WHEN negative THEN
    DBMS_OUTPUT.PUT_LINE(' –VE MARKS');
END;
/
```

To run PL/SQL program

From SQL*Plus Environment

SQL> start student
Or
SQL> @student

From iSQL*Plus Environment

Use **Execute** button

Program 2

Write a program to get the salary of an employee

```
DECLARE
        Vempno    emp.empno%TYPE;
        Vsal          emp.sal%TYPE;
BEGIN
        Vempno   :=  &empno;
        SELECT sal into vsal from emp where empno = Vempno;
        DBMS_OUTPUT.PUT_LINE('SALARY = ' || Vsal);
EXCEPTION
        WHEN  no_data_found THEN
             DBMS_OUTPUT.PUT_LINE('Employee not found…');
END;
/
```

a)Check what happened when we input a 5 digit number
b)Check what happens when we input employee number in single quotes
c)Check what happens when we input alpha-numeric information
d)Check what happens when there is a duplicate empno

To handle all the above errors, we can modify the above program as

```
DECLARE
        Vempno    emp.empno%TYPE;
       Vsal          emp.sal%TYPE;
BEGIN
       Vempno   :=  &empno;
       SELECT sal into vsal from emp where empno = Vempno;
       DBMS_OUTPUT.PUT_LINE('SALARY = ' || Vsal);
EXCEPTION
       WHEN  no_data_found THEN
           DBMS_OUTPUT.PUT_LINE('Employee not found…');
        WHEN others then
          DBMS_OUTPUT.PUT_LINE(SQLCODE || SQLCODE);
END;
/
```

Note :- SQLCODE and SQLERRM are called as Error Trapping functions.
SQLCODE displays error number (ORA-00001)
SQLERRM displays SQL error message

Exercise

1) Write a program to maintain employee details with
   Empno, Ename, basic, da, hra, gross, pf, net

   a) Generate Empno automatically
   b) Input name and basic salary
   c) Calculate da, hra, gross, pf and net
   d) Assume da and hra some % on basic
   e) Gross = basic + da + hra
   f) Pf is 10% on gross
   g) Net is gross – pf

Handle required exceptions

Summary

In this lesion, you should have learned that:

- Exception types:
  - Predefined Oracle server error
  - Nonpredefined Oracle server error
  - User-defined error
- Exception trapping
- Exception handling
  - Trap the exception within the PL/SQL block.
  - Propagate the exception

LOOPS

1) Write a program to print the numbers in the format

```
1  2  3
4  5  6
7  8  9
```

2) Write a program to accept a string only with lowercase letters

1) Solution

```
begin
  dbms_output.new_line;
  for i in 1 .. 8
  loop
    for j in 1 .. 8
    loop
      dbms_output.put(j||' ');
    end loop;
    dbms_output.new_line;
 end loop;
end;
```

3.0 Cursors

What is a Cursor?

Oracle uses work area to execute SQL commands and store processing information. PL/SQL allows you to access this area through a name using a cursor.

Whenever you issue a SQL statement, the Oracle Server opens an area of memory in which the command is parsed and executed. This area is called a cursor.

When you execute a SQL statement from PL/SQL, the Oracle RDBMS assigns a private work area for that statement. This work area contains information about the SQL statement and the set of data returned or affected by that statement.

The PL/SQL cursor is a mechanism by which you can name that work area and manipulate the information within it. In its simplest form, you can think of a cursor as a pointer into a table in the database.

Cursors are of two types in PL/SQL

- Implicit cursor
- Explicit cursor

Implicit cursor

PL/SQL declares a cursor implicitly for all SQL data manipulation statements, including queries that return only one row. However, for queries that return more than one row, you must declare an explicit cursor or use a cursor FOR loop. The name of the implicit cursor is SQL. You can directly use the cursor without any declaration.

Explicit Cursor

The set of rows returned by a query can consist of zero, one or multiple rows, depending on how many rows meet your search criteria. When a query returns multiple rows, you can explicitly declare a cursor to process the rows. **The set of rows returned by a multiple-row query is called the active set.** ***It is manipulated just like a file in programming languages.***

Explicit cursor functions

- Can process beyond the first row returned by the query, row by row.
- Keep track of which row is currently being processed.
- Allow the programmer to manually control them in the PL/SQL block.

Note: The fetch for an implicit cursor is an array fetch, and the existence of a second row still raises the TOO_MANY_ROWS exception. Furthermore, you can use explicit cursors to perform multiple fetches and to re-execute parsed queries in the work area.

Handling Explicit Cursor

Explicit cursor is a name used to refer to an area where you can place multiple rows retrieved by SELECT

STEPS

The following are the required steps to process an explicit cursor.

- Declare the cursor in declare section
- Open the cursor using OPEN
- Fetch rows from the cursor FETCH
- Close the cursor after the process is over using CLOSE

How a cursor works?

Let us understand the cursor, with C File. To make any changes in the file first we have to open it (FOPEN () method). Once it is opened, it is placed in the memory and a file pointer identifies this memory area (FP). To read a particular line of information, we use a method (FREAD ()). The read information is stored into a variable. The variable size is equal to the size of the information that you are reading. To read each and every line from the file, we have to use a loop.
This loop is performed until it identifies EOF. Then we have to close the file (FCLOSE () method), to free the memory space.

Assume that the file is there in the common share folder(in the server). Reading each line from the server takes more time as well as we have to interact with the server repeatedly, which increases the network traffic and reduces the efficiency.

Similarly, assume that your server is a Oracle server and the file is a database. Every time we have to interact with server to get one row from the database. For getting multiple records, each time we have to interact with database, which reduces the efficiency. To over come this situation, we are defining a cursor, through which we can get the required information into the memory and that memory area is identified with cursor name (Like a file pointer).

In the place of fopen () method we are using OPEN () method. It places all the required data into the memory.

We are using FREAD () method for reading each line of information from the file. In the similar way we use FETCH method to get each record from memory.

After fetching one record, it should be placed into a variable. So, we have to create a variable whose size is equal to the size of the information that we are fetching.

Then we have to use a loop to read each and every record until nothing is there to fetch.

Then close the cursor using CLOSE () method.

In more technical terms, a cursor is the name for a structure in memory, called a *private SQL area*, which the server allocates at runtime for each SQL statement. This memory area contains, among other things, a parsed version of the original SQL statement.

If the host program uses any variables in the SQL statement, the cursor also contains the memory addresses of these variables.

When you put SELECT statements into your PL/SQL, there are two ways to deal with the data. You can use the SELECT INTO, as seen in the previous section, in which case you are using an *implicit cursor* ("implicit" because you don't refer to it specifically in your code; Oracle manages implicit cursors automatically). The second way gives you more direct control over the creation, naming, and use of cursors associated with your SELECTs. These cursors are called explicit cursors.

How to Code an Explicit Cursor

An explicit cursor in PL/SQL is one with which you "explicitly" code each of the following steps:

1. Declare the cursor in declare section
2. Open the cursor using OPEN
3. Fetch rows from the cursor using FETCH
4. Close the cursor after the process is over using CLOSE.

Declare a Cursor

A cursor is declared in DECLARE section using CURSOR statement.

Syntax

```
Cursor <cursorname> is
        Select <column(s)> from <tablename> where
        <Condition>;
```

Example

```
Cursor emp_cur is
  Select empno, ename, job, sal from EMP where empno >= 7521;
```

Note: No data is actually retrieved at the time of cursor declaration. Data is placed in the cursor when cursor is opened.

Opening a cursor using OPEN command

OPEN statement is used to execute the query associated with the cursor and place the result into cursor.

Syntax

```
OPEN cursor_name;
```

Example

```
Open emp_cur;
```

When a cursor is opened, all the rows retrieved by SELECT, given at the time of cursor declaration, are placed in the cursor.

Fetching rows from a cursor using FETCH command

Once cursor is opened using OPEN, cursor has a set of rows, which can be fetched using FETCH statement.

Syntax

```
FETCH cursor_name INTO variable1, variable2, …
```

For each column in the cursor there should be a corresponding variable in FETCH statement.

FETCH statement is to be repeatedly executed to fetch all the rows of the cursor.

Closing a Cursor using CLOSE command

CLOSE statement is used to close after the cursor is processed.

Syntax

```
CLOSE cursor_name
```

Example

```
CLOSE emp_cur;
```
Program

Write a program to test the cursor

```
SET SERVEROUTPUT ON   -- SQL*plus Environment command
DECLARE

    Cursor emp_cur is
    Select empno, ename, job, sal from EMP where empno >= 7521;
    Emp_rec  emp_cur%rowtype;

BEGIN

   /* open the cursor */

    Open emp_cur;

  /* fetch a record from cursor */

  FETCH emp_cur into emp_rec;

DBMS_OUTPUT.PUT_LINE(emp_rec.empno || emp_rec.ename|| emp_rec.sal);

-- closing the cursor

  CLOSE emp_cur;

END;
/
```

Analysis

This program reads and prints only one record from cursor

Program

To read each and every record from the cursor

```
SET SERVEROUTPUT ON
DECLARE

    Cursor emp_cur is
    Select empno, ename, job, sal from EMP where empno >= 7521;
    Emp_rec  emp_cur%rowtype;

BEGIN

   /* open the cursor */

    Open emp_cur;

  /* fetch all the records of the cursor one by one */

LOOP
  FETCH emp_cur into emp_rec;

/*

      Exit loop if reached end of cursor
      NOTFOUND is the cursor attribute

*/
exit when emp_cur%NOTFOUND;
DBMS_OUTPUT.PUT_LINE (emp_rec.empno || emp_rec.ename|| emp_rec.sal);

END LOOP;
-- closing the cursor

  CLOSE emp_cur;

END;
/
```

Passing parameters

Program

```
SET SERVEROUTPUT ON
DECLARE

    Cursor emp_cur(v_empno number) is    -- formal parameter
    Select empno, ename, job, sal from EMP where empno >= v_empno;
    Emp_rec  emp_cur%rowtype;
   V_eno  emp.empno%type;
BEGIN
    /* input the employee number */

     v_eno  := &empno;

   /* open the cursor */

    Open emp_cur(v_eno);  -- Actual argument

  /* fetch all the records of the cursor one by one */

LOOP
  FETCH emp_cur into emp_rec;

/*

      Exit loop if reached end of cursor
      NOTFOUND is the cursor attribute

*/
exit when emp_cur%NOTFOUND;
DBMS_OUTPUT.PUT_LINE (emp_rec.empno || emp_rec.ename|| emp_rec.sal);

END LOOP;
-- closing the cursor

  CLOSE emp_cur;

END;
/
```

Note : We can pass any number of arguments

Program

```
SET SERVEROUTPUT ON
DECLARE

    Cursor emp_cur(v_empno_from number, v_empno_to number ) is
         -- formal parameter and called as input arguments
    Select empno, ename, job, sal from EMP where empno between
    v_empno_from and v_empno_to;

    Emp_rec  emp_cur%rowtype;
   V_eno  emp.empno%type;
BEGIN
    /* input the employee number */

     v_empno_from  := &start_empno;
     v_empno_to      := &end_empno;
   /* open the cursor */

    Open emp_cur(v_empno_from,v_empno_to);  -- Actual argument

  /* fetch all the records of the cursor one by one */

LOOP
  FETCH emp_cur into emp_rec;

/*

      Exit loop if reached end of cursor
      NOTFOUND is the cursor attribute

*/
exit when emp_cur%NOTFOUND;
DBMS_OUTPUT.PUT_LINE (emp_rec.empno || emp_rec.ename|| emp_rec.sal);

END LOOP;
-- closing the cursor

  CLOSE emp_cur;

END;
/
```

Declaring Multiple Cursors

```
SET SERVEROUTPUT ON
DECLARE

   Cursor dept_cur is
         Select * from dept;
   Cursor emp_cur(v_deptno IN number) is
          Select empno,ename,sal from EMP where deptno = v_deptno;

   Dept_rec  dept_cur%rowtype;
   Emp_rec  emp_cur%rowtype;
 BEGIN
 OPEN dept_cur;
  LOOP
     FETCH dept_cur into dept_rec;
     EXIT when dept_cur%NOTFOUND:
      DBMS_OUTPUT.put_line ('Employees working Under  '  || dept_rec.deptno);
             OPEN emp_cur (dept_rec.deptno);
                LOOP
                   FETCH emp_cur into emp_rec;
                   EXIT when emp_cur%NOTFOUND;
                     DBMS_OUTPUT.PUT_LINE (emp_rec.empno
                                 || emp_rec.ename||emp_rec.job);
               END LOOP;
          CLOSE emp_cur;
   END LOOP:
  CLOSE dept_cur;
END:
/
```

Cursor Attributes

Cursor attributes allow to get information regarding cursor. For example, you can get the number of rows fetched so far from a cursor using ROWCOUNT

Use the following syntax to access cursor attributes

Cursor_name%Attribute

The following is the list of available cursor attributes:

Attribute	**Data type**	**Significance**	**Recommended time to use**
FOUND	BOOLEAN	TRUE if most recent fetch found a row in the table; otherwise FALSE	After opening and fetching from the cursor but before closing it (will be NULL before first fetch)
NOTFOUND	BOOLEAN	This the just logical inverse of FOUND	Same as above
ROWCOUNT	BOOLEAN	Number of Rows fetched so far	Same as above (except it will be zero before the first fetch)

In addition to those cursor attributes, there are some less-commonly used cursor attributes that you might see from time to time. They include:

ISOPEN Returns TRUE or FALSE depending on whether cursor_name is open.

Program

```
SET SERVEROUTPUT ON
DECLARE
    Cursor emp_cur is
    Select empno, ename, job, sal from EMP where empno >= 7521;
    Emp_rec  emp_cur%rowtype;
BEGIN
   If not emp_cur%ISOPEN then
    Open emp_cur;
   End if;
LOOP
  FETCH emp_cur into emp_rec;
  If emp_cur%FOUND then
  DBMS_OUTPUT.put_line('No of rows effected ' || emp_cur%ROWCOUNT);
 Else
   DBMS_OUTPT.put_line('END Of file ');
END IF;
exit when emp_cur%NOTFOUND;
DBMS_OUTPUT.PUT_LINE (emp_rec.empno || emp_rec.ename|| emp_rec.sal);
END LOOP;
  CLOSE emp_cur;
END;
```

Cursor attributes can be used with both implicit and explicit cursors.

For example, the following shows how to use cursor attribute with implicit cursor:

```
Begin

    Update EMP set sal = 1200 where ename like '%E%';

     /* If more than 5 rows are effected then rollback updation */

if SQL%ROWCOUNT > 5 then
       Rollback;
else
       commit;
 end if;
End;
/
```

Note: -

1) Except ISOPEN, all other attributes are implicit cursor attributes

2) When %ROWCOUNT is used with implicit cursor, it returns number of rows effected by UPDATE, INSERT and DELETE commands. And when used with explicit cursors, it returns the number of rows fetched so far from the cursor.

Cursor For Loop

In order to process a cursor, you can use cursor FOR loop to automate the following steps.

- Opening cursor
- Fetching rows from the cursor
- Terminating loop when all rows in the cursor are processed
- Closing cursor

Syntax

```
FOR  rowtype_variable IN cursor_name
LOOP
      Statements;
END LOOP;
```

Example

```
SET SERVEROUTPUT ON
declare
  cursor c1 is
   select empno,ename from emp;
begin
    for c in c1
    loop
       dbms_output.put_line(c.empno || c.ename);
    end loop;
end;
/
```

The following are the important steps in the above program:

- Cursor c1 is automatically opened by cursor for loop
- C is declared automatically as :
  C c1%ROWTYPE;

But C is available only inside the cursor for loop. It contains the same columns as the cursor. In order to access a column of the current row of the cursor, in the cursor for loop, use the format:

Rowtype_variable.columnname

- Statements in the cursor for loop are executed once for each row of the cursor. And for each row of the cursor, row is copied into rowtype_variable.
- Loop is terminated once end of cursor is reached. And cursor is closed.

Note : If parameters are to be passed to cursor, give the values after the name of the cursor.

```
declare
  cursor c1 (n number) is
   Select empno, ename from EMP where empno >= n;
begin
    for c in c1(7521)
    loop
       dbms_output.put_line (c.empno || c.ename);
    end loop;
end;
```

/

For Update Of and Current Of

By default, Oracle locks rows while updating and deleting rows. But it is possible to override default locking by using FOR UPDATE.

FOR UPDATE clause can be used with SELECT while declaring cursor to lock all records retrieved by cursor to make sure they are not changed before we update or delete them. As Oracle automatically locks rows for you. FOR UPDATE clause is required only when you want to lock rows before the update or delete – at the time of opening cursor.

CURRENT OF clause can be used to refer to the current row in the cursor.

Note: FOR UPDATE must be given if you want to use CURRENT OF clause to refer to current row in the cursor.

Exercise

Write a program to Modify a particular entry in the passbook and update the corresponding balances.(Handle suitable exceptions).

PASSBOOK

SNO	TRNDATE	TRNTYPE	AMOUNT	BALANCE
1	SYSDATE	D	30000	30000
2	SYSDATE	W	10000	20000
3	SYSDATE	W	2000	18000
4	SYSDATE	D	5000	23000
5	SYSDATE	W	3000	20000
6	SYSDATE	D	8000	28000

In the above table, if we modify the amount in one particular transaction, Write a program to modify the corresponding balances.

Handle suitable exceptions, to accept only a positive amount and in no case, balance should become negative.

Summary

A cursor is always used by PL/SQL to execute single-row queries and DML command. But in order to use multi-row query, you have to use an explicit cursor. An explicit cursor contains a row-set, which is retrieved by multi-row query.

Specially designed cursor FOR loop for cursors make cursor handling very easy and automatic. FOR UPDATE clause is used to override default locking and CURRENT OF is used to refer to current record in the cursor.

Subprograms

Subprograms are named PL/SQL blocks that can accept parameters and be invoked from a calling environment.

PL/SQL has two types of subprograms, ***procedures*** **and** ***functions*****.**

What Is a Procedure?
• A procedure is a type of subprogram that performs an action.
• A procedure can be stored in the database, as a schema object, for repeated execution.

Definition of a Procedure

A procedure is a named PL/SQL block that can accept parameters (sometimes referred to as arguments), and be invoked. Generally speaking, you use a procedure to perform an action. A procedure has a header, a declaration section, an executable section, and an optional exception-handling section.

A procedure can be compiled and stored in the database as a schema object. Procedures promote reusability and maintainability. When validated, they can be used in any number of applications. If the requirements change, only the procedure needs to be updated.

Creating Procedures

Objectives

After completing this lesson, you should be able to do the following:

- **Distinguish anonymous PL/SQL blocks from named PL/SQL blocks (subprograms)**
- **List the benefits of using subprograms**
- **List the different environments from which subprograms can be invoked**

Objectives

After completing this lesson, you should be able to do the following:

- **Describe PL/SQL blocks and subprograms**
- **Describe the uses of procedures**
- **Create procedures**
- **Differentiate between formal and actual parameters**
- **List the features of different parameter modes**
- **Create procedures with parameters**
- **Invoke a procedure**
- **Handle exceptions in procedures**
- **Remove a procedure**

Lesson Aim

In this lesson, you learn the difference between anonymous PL/SQL blocks and subprograms. You also learn to create, execute, and remove procedures.

Overview of Subprograms

A subprogram:

- **Is a named PL/SQL block that can accept parameters and be invoked from a calling environment?**
- **Is of two types:**
  - **A procedure that performs an action**
  - **A function that computes a value**
- **Is based on standard PL/SQL block structure**
- **Provides modularity, reusability, extensibility, and maintainability.**
- **Provides easy maintenance, improved data security and integrity, improved performance, and improved code clarity**

A subprogram is based on standard PL/SQL structure that contains a declarative section, an executable section, and an optional exception-handling section.

A subprogram can be compiled and stored in the database. It provides modularity, extensibility, reusability, and maintainability.

Modularization is the process of breaking up large blocks of code into smaller groups of code called modules. After code is modularized, the modules can be reused by the same program or shared by other programs. It is easier to maintain and debug code of smaller modules than a single large program. Also, the modules can be easily extended for customization by incorporating more functionality, if required, without affecting the remaining modules of the program.
Subprograms provide easy maintenance because the code is located in one place and hence any modifications required to the subprogram can be performed in this single location. Subprograms provide improved data integrity and security. The data objects are accessed through the subprogram and a user can invoke the subprogram only if appropriate access privilege is granted to the user.

Benefits of Sub Programs

- Easy maintenance
- Improved data security and Integrity
- Improved performance
- Improved code clarity

Procedures and functions have many benefits in addition to modularizing application development:

• Easy maintenance: Subprograms are located in one location and hence it is easy to:
  – Modify routines online without interfering with other users
  – Modify one routine to affect multiple applications
  – Modify one routine to eliminate duplicate testing

• Improved data security and integrity
  – Controls indirect access to database objects from nonprivileged users with security privileges. As a subprogram is executed with its definer's right by default, it is easy to restrict the access privilege by granting a privilege only to execute the subprogram to a user.

  - Ensures that related actions are performed together, or not at all.

• Improved performance

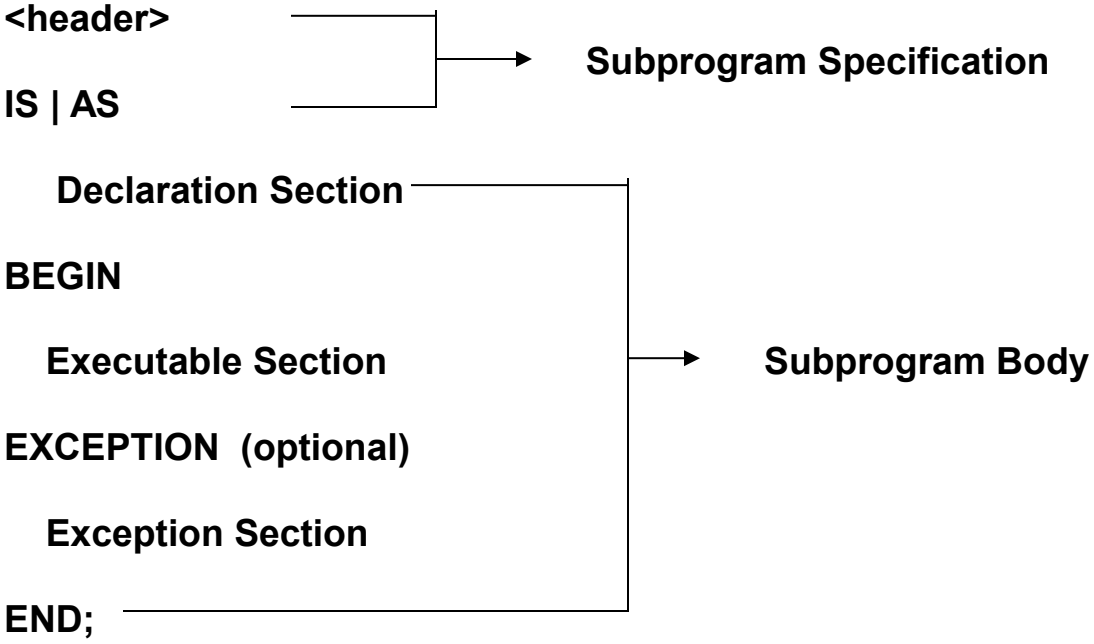
  – After a subprogram is compiled, the parsed code is available in the shared SQL area of the server and subsequent calls to the subprogram use this parsed code. This avoids reparsing for multiple users.

  – Avoids PL/SQL parsing at run time by parsing at compile time

    — Reduces the number of calls to the database and decreases network traffic by bundling commands.

    — Improves code clarity: Using appropriate identifier names to describe the action of the routines reduces the need for comments and enhances the clarity of the code.

Block structure for PL/SQL Subprograms



Subprogram Specification

• The header is relevant for named blocks only and determines the way that the program unit is called or invoked.

The header determines:

– The PL/SQL subprogram type, that is, either a procedure or a function
– The name of the subprogram
– The parameter list, if one exists
– The RETURN clause, which applies only to functions

• The IS or AS keyword is mandatory.

Subprogram Body

- The declaration section of the block between IS|AS and BEGIN. The keyword DECLARE that is used to indicate the start of the declaration section in anonymous blocks is not used here.

- The executable section between the BEGIN and END keywords is mandatory, enclosing the body of actions to be performed. There must be at least one statement existing in this section. There should be at least one NULL; statement, which is considered an executable statement.

- The exception section between EXCEPTION and END is optional. This section traps predefined error conditions. In this section, you define actions to take if the specified error condition arises.

Syntax for Creating Procedures

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter1 [mode1] datatype1,
parameter2 [mode2] datatype2,. . .)]
IS|AS
PL/SQL Block;
```

- **The REPLACE option indicates that if the procedure exists, it will be dropped and replaced with the new version created by the statement.**

- **PL/SQL block starts with either BEGIN or the declaration of local variables and ends with either END or END procedure_name.**

Syntax Definitions

You create new procedures with the CREATE PROCEDURE statement, which may declare a list of parameters and must define the actions to be performed by the standard PL/SQL block. The CREATE clause enables you to create stand-alone procedures, which are stored in an Oracle database.

• PL/SQL blocks start with either BEGIN or the declaration of local variables and end with either END or END procedure_name. You cannot reference host or bind variables in the PL/SQL block of a stored procedure.

• The REPLACE option indicates that if the procedure exists, it will be dropped and replaced with the new version created by the statement.
• You can not restrict the size of the data type in the parameters.

Parameter	Description
Procedure_Name	Name of the procedure
Parameter	Name of PL/SQL variable whose value is passed to or populated by the calling environment, or both, depending on the mode being used
Mode	Type of argument IN (default) OUT IN OUT
Data type	Data type of argument – can be any SQL/ PLSQL data type. Can be of %TYPE, %ROWTYPE, or any scalar or composite data type. **You can not restrict the size of the data type in the parameters**.
PL/SQL block	Procedural body that defines the action performed by the procedure.

Formal versus Actual Parameters

- **Formal parameters: variables declared in the parameter list of a subprogram specification.**

Example:

```
CREATE PROCEDURE raise_sal(p_id NUMBER,
  p_amount NUMBER)
 ...
END raise_sal;
```

- **Actual parameters: variables or expressions referenced in the parameter list of a subprogram call.**

Example:

```
raise_sal(v_id, 2000)
```

Procedural Parameter Modes

You can transfer values to and from the calling environment through parameters. Select one of the three modes for each parameter: IN, OUT, or IN OUT.

Attempts to change the value of an IN parameter will result in an error.

Note: DATATYPE can be only the %TYPE definition, the %ROWTYPE definition, or an explicit data type with no size specification.

Type of Parameter Description

IN (default) passes a constant value from the calling environment into the procedure.

OUT passes a value from the procedure to the calling environment.

IN OUT Passes a value from the calling environment into the procedure and a possibly different value from the procedure back to the calling environment using the same parameter

Creating Procedures with Parameters

IN	OUT	IN OUT
Default Mode	**Must be specified**	**Must be specified**
Value is passed into subprogram	**Returned to Calling Environment**	**Passed into subprogram; returned to calling environment.**
Formal parameters acts as a constant	**Uninitialized variable**	**Initialized variable**
Actual parameter can be a literal, expression, constant, or initialized variable	**Must be a variable**	**Must be a variable**
Can be assigned a default value	**Cannot be assigned a default value**	**Cannot be assigned a default value**

IN Parameters: Example

```
CREATE OR REPLACE PROCEDURE raise_salary
(p_id IN emp.empno%TYPE)
IS
  Vsal  emp.sal%type;
BEGIN
Select sal into vsal from emp WHERE empno = p_id;
Dbms_output.put_line(vsal);
END raise_salary;
/
```

IN Parameters: Example

The example shows a procedure with one IN parameter. Running this statement in *i*SQL*Plus creates the RAISE_SALARY procedure. When invoked, RAISE_SALARY accepts the parameter for the employee ID and gets the Salary of that employee.

To invoke a procedure in *i*SQL*Plus, use the EXECUTE command.

```
EXECUTE raise_salary (176)
```

To invoke a procedure from another procedure, use a direct call. At the location of calling the new procedure, enter the procedure name and actual parameters.

```
raise_salary (176);
```

IN parameters are passed as constants from the calling environment into the procedure. Attempts to change the value of an IN parameter result in an error.

OUT Parameters: Example

```
CREATE OR REPLACE PROCEDURE query_emp
(p_id  emp.empno%TYPE,
p_name OUT emp.ename%TYPE,
p_salary OUT emp.sal%TYPE,
p_comm OUT emp.comm%TYPE)
IS
BEGIN
SELECT  ename, sal, comm. INTO p_name, p_salary, p_comm
FROM emp WHERE empno = p_id;
END query_emp;
/
```

OUT Parameters: Example (continued)

Run the script file shown in the slide to create the QUERY_EMP procedure. This procedure has four formal parameters. Three of them are OUT parameters that return values to the calling environment.

The procedure accepts an EMPNO value for the parameter P_ID. The name, salary, and commission values corresponding to the employee ID are retrieved into the three OUT parameters whose values are returned to the calling environment.

Viewing OUT Parameters

```
VARIABLE g_name VARCHAR2(25)
VARIABLE g_sal NUMBER
VARIABLE g_comm NUMBER
EXECUTE query_emp(171, :g_name, :g_sal, :g_comm)
PRINT g_name
```

How to View the Value of OUT Parameters with *i*SQL*Plus

1. Run the SQL script file to generate and compile the source code.

2. Create host variables in *i*SQL*Plus, using the VARIABLE command.

3. Invoke the QUERY_EMP procedure, supplying these host variables as the OUT parameters.

Note the use of the colon (:) to reference the host variables in the EXECUTE command

4. Variables which are used with **:** are called as binded variables. The meaning of binded is, getting the value that we are getting from PI/SQL environment to SQL environment.

.
5. To view the values passed from the procedure to the calling environment, use the PRINT command.

The example shows the value of the G_NAME variable passed back to the calling environment. The other variables can be viewed, either individually, as above, or with a single PRINT command.

```
PRINT g_name g_sal g_comm
```

Do not specify a size for a host variable of data type NUMBER when using the VARIABLE command. A host variable of data type CHAR or VARCHAR2 defaults to a length of one, unless a value is supplied in parentheses.

IN OUT Parameters

Example

Create a procedure with an IN OUT parameter to accept a character string containing 10 digits and return a phone number formatted as (800) 633-0575.

Calling environment FORMAT_PHONE procedure
'8006330575' '(800)633-0575' p_phone_no

```
CREATE OR REPLACE PROCEDURE format_phone
(p_phone_no IN OUT VARCHAR2)
IS
BEGIN
p_phone_no := '(' || SUBSTR(p_phone_no,1,3) ||
')' || SUBSTR(p_phone_no,4,3) ||
'-' || SUBSTR(p_phone_no,7);
END format_phone;
/
```

Using IN OUT Parameters

With an IN OUT parameter, you can pass values into a procedure and return a value to the calling environment. The value that is returned is the original, an unchanged value, or a new value set within the procedure.

An IN OUT parameter acts as an initialized variable.

Run the statement to create the FORMAT_PHONE procedure.

Viewing IN OUT Parameters

```
VARIABLE g_phone_no VARCHAR2(15)
BEGIN
:g_phone_no := '8006330575';
END;
/
PRINT g_phone_no
EXECUTE format_phone (:g_phone_no)
PRINT g_phone_no
```

Exercise

1. Write a procedure to get the salary of an employee
2. Write a procedure to get the name and hiredate of an employee
3. Write a procedure to get all the details of a particular employee
4. Write a procedure to delete particular employee
5. Write a procedure to encrypt and decrypt your password
6. Write a procedure to get the total salary of all the managers
7. Write a procedure to get the name and salary of the highest paid employee

Creating Functions

Objectives

After completing this lesson, you should be able to do the following:

- **Describe the uses of functions**
- **Create stored functions**
- **Invoke a function**
- **Remove a function**
- **Differentiate between a procedure and a function**

Lesson Aim

In this lesson, you will learn how to create and invoke functions.

Overview of Stored Functions

- A function is a named PL/SQL block that returns a value.
- A function can be stored in the database as a schema object for repeated execution.
- A function is called as part of an expression.

Stored Functions

A function is a named PL/SQL block that can accept parameters and be invoked.

Generally speaking,

You use a function to compute a value. Functions and procedures are structured alike. A function must return a value to the calling environment, whereas a procedure returns zero or more values to its calling environment. Like a procedure, a function has a header, a declarative part, an executable part, and an optional exception-handling part. A function must have a RETURN clause in the header and at least one RETURN statement in the executable section.

Functions can be stored in the database as a schema object for repeated execution. A function stored in the database is referred to as a stored function.

```
CREATE [OR REPLACE] FUNCTION function_name
[(parameter1 [mode1] datatype1,
parameter2 [mode2] datatype2,. . .)]
RETURN datatype
IS|AS
PL/SQL Block;
The PL/SQL block must have at least one RETURN
statement.
```

Example

```
CREATE OR REPLACE FUNCTION get_sal
(p_id  emp.empno%TYPE)
RETURN NUMBER
IS
v_salary emp.sal%TYPE ;
BEGIN
SELECT sal INTO v_salary FROM emp WHERE empno = p_id;
RETURN v_salary;
END get_sal;
/
```

Executing Functions

• Invoke a function as part of a PL/SQL expression.

• Create a variable to hold the returned value.

• Execute the function. The variable will be

populated by the value returned through a RETURN statement.

Example

```
VARIABLE g_salary NUMBER
EXECUTE :g_salary := get_sal(117)
PRINT g_salary
```

Exercise

1. Write a function to get the retirement date of a particular employee
2. Write a function to calculate income tax
3. Write a function to get the details of a particular employee
4. Write a program to maintain cashbook
   Vchno vchdate trntype amount balance

How Procedures and Functions Differ

1. You create a procedure to store a series of actions for later execution. A procedure can contain zero or more parameters that can be transferred to and from the calling environment, but a procedure does not have to return a value.

2. You create a function when you want to compute a value, which must be returned to the calling environment. A function can contain zero or more parameters that are transferred from the calling environment. Functions should return only a single value, and the value is returned through a RETURN statement.

3. Functions used in SQL statements cannot have OUT or IN OUT mode parameters.

Procedure to Encrypt and Decrypt a password

```
create or replace procedure encry(u varchar2,p varchar2)
is
   x varchar2(100);
  begin
   for i in 1 .. length(p)
   loop
      x := x || chr(ascii(substr(upper(p),i,1)) - 42) ;
   end loop;
insert into logram values(u,x);
   dbms_output.put_line(x);
  end;
/



create or replace procedure decry(u1 varchar2)
is
   nm varchar2(20);
   x varchar2(20);

begin
   select p into  nm from logram where u=u1;
   for i in 1 .. length(nm)
   loop
       x := x || chr(ascii(substr(nm,i,1))+42);
   end loop;

   dbms_output.put_line(x);
end;
/
```

PACKAGES

What is A Package?

Packages are containers of related procedures, functions and variables.

When we create different procedures and functions, they are stored as independent objects. To group these objects together, we are creating packages.

Each package contains the following parts:

- Package specification
- Package body

Package Specification

Contains all declarations for variable, cursor, procedures and functions that are to be made public. All public objects of package are visible outside the package.

Syntax

```
CREATE OR REPLACE PACKAGE  package_name
IS
   /* declare public objects of package */
END;
```

Package Body

Defines all the objects of the package. Objects declared in the specification are called as public objects and the objects directly defined in the body without declaring in the specification, are called as PRIVATE members.

Syntax

The package body:

```
CREATE or REPLACE PACKAGE BODY package_name
IS
[ declarations of variables and types ]
[ specification and SELECT statement of cursors ]
[ specification and body of modules ]
[ BEGIN
executable statements ]
[ EXCEPTION
exception handlers ]
END [ package_name ];
```

In the body you can declare other variables, but you do not repeat the declarations in the specification.

The body contains the full implementation of cursors and modules. In the case of a cursor, the package body contains both specification and SQL statement for the cursor. In the case of a module the package body contains both the specification and body of the module.

The BEGIN keyword indicates the presence of an execution or initialization section for the package. This section can also optionally include an exception section.

As with a procedure or function, you can add the name of the package, as a label, after the END keyword in both the specification and package

The body of a package can contain

- Procedures declared in the package specification
- Functions declared in the package specification
- Definitions of cursors declared in the package specification
- Local procedures and functions, not declared in the package specification.

- Local variables

Program

```
CREATE OR REPLACE PACKAGE SAMPLEPAK
IS
   PROCEDURE PROC1( N  NUMBER, N1 OUT NUMBER);
   FUNCTION     FUN1(N NUMBER) RETURN NUMBER;
 END;
/
CREATE OR REPLACE PACKAGE BODY SAMPLEPAK
IS
  PROCEDURE PROC1(N NUMBER, N1 OUT NUMBER)
  IS
  BEGIN
            N1 := N * 5;
   END PROC1;
  FUNCTION FUN1(N NUMBER) RETURN NUMBER
  IS
    N1 NUMBER;
  BEGIN
           N1  := N * 2;
           RETURN N1;
  END FUN1;
END SAMPLEPAK;
/
```

Execution

```
VARIABLE N NUMBER
EXECUTE SAMPLEPAK.PROC1(5,:N)
 PRINT N


EXECUTE :N := SAMPLEPAK.FUN1(4)
PRINT N
```

Program to define private member

```
CREATE OR REPLACE PACKAGE SAMPLEPAK
IS
    PROCEDURE PROC1( N  NUMBER, N1 OUT NUMBER);
    FUNCTION     FUN1(N NUMBER) RETURN NUMBER;
 END;
/
CREATE OR REPLACE PACKAGE BODY SAMPLEPAK
IS
   PROCEDURE TEST     -- private member definition
   IS
   BEGIN
         DBMS_OUTPUT.PUT_LINE('I AM A PRIVATE MEMBER');
  END;
   PROCEDURE PROC1(N NUMBER, N1 OUT NUMBER)
   IS
   BEGIN
             TEST;     -- private member called
              N1 := N * 5;
    END PROC1;
   FUNCTION FUN1(N NUMBER) RETURN NUMBER
   IS
      N1 NUMBER;
   BEGIN
             N1  := N * 2;
             RETURN N1;
   END FUN1;
END SAMPLEPAK;
/
```

Execution

```
VARIABLE N NUMBER
EXECUTE SAMPLEPAK.PROC1(5,:N)
 PRINT N


EXECUTE :N := SAMPLEPAK.FUN1(4)
PRINT N
```

Note :- A private member can not be accessed by referring package object. They are called only through public members of the package object.

Program to test Polymorphism

Program

```
CREATE OR REPLACE PACKAGE POLYPACK
IS
    PROCEDURE PROC1( N  NUMBER, N1 OUT NUMBER);
    PROCEDURE   PROC1  (X VARCHAR2,Y VARCHAR2,Z OUT VARCHAR2);
 END;
/
CREATE OR REPLACE PACKAGE BODY POLYPACK
IS
   PROCEDURE PROC1(N NUMBER, N1 OUT NUMBER)
   IS
   BEGIN
              N1 := N * 5;
    END PROC1;
   PROCEDURE PROC1(X VARCHAR2,Y VARCHAR2, Z OUT VARCHAR2)
   IS
   BEGIN
           Z :=  CONCAT(X,Y);
   END PROC1;
END POLYPACK;
/
```

Execution

```
VARIABLE N NUMBER
 VARIABLE  ST VARCHAR2(100)
EXECUTE POLYPACK.PROC1(5,:N)
 PRINT N


EXECUTE  POLYPACK.PROC1('RAVI',' KIRAN',:ST)
PRINT ST
```

Creating Database Triggers

Objectives

After completing this lesion, you should be able to do the following:

- **Describe different types of triggers**
- **Describe database triggers and their use**
- **Create Database triggers**
- **Describe database trigger firing rules**
- **Remove database triggers**

Trigger - Levels

These Triggers are written at three different levels

Schema **TABLE** **ROW**

A Schema level trigger is a trigger which is written at database level or user level.. These triggers can be written by DBA only.

Any user can write table and ROW level triggers. Table level triggers are ment for providing security at object (Table) level.

Row level triggers are ment for validations

Types of Triggers

A trigger:

- **is a PL/SQL block or a PL/SQL procedure associated with a table, view, schema, or the database.**
- **Executes implicitly whenever a particular event takes place**
- **Can be either**
  - **Application trigger : Fires whenever an event occurs with a particular application.**
  - **Database trigger: Fires whenever a data event (such as DML) or system event (such as logon or shutdown) occurs on a schema or database.**

Types of Triggers

- ROW Triggers and Statement Triggers
- BEFORE or AFTER Triggers
- INSTRAD-OF triggers
- Triggers on System events and User Events

Row Triggers : A row trigger is fired each time the table is affected by triggering statement. For example, if an UPDATE statement updates multiple rows of a table, a row trigger is fired once for each row affected by the UPDATE statement. If a triggering statement affects no rows, a row trigger is not executed at all.

A Row trigger is fired once for each row affected by the command. These triggers are used to check for the validity of the data in the triggering statements and rows affected.

Statement Triggers A statement trigger is fired once on behalf of the triggering statement, regardless of the number of rows in the table that the triggering statement affects (even no rows are affected). For example, if a DELETE statement deletes several rows from a table, a statement level DELETE trigger is fired only once, regardless of how many rows are deleted from the table.

BEFORE and AFTER Triggers When defining a trigger, you can specify the trigger timing- whether the trigger action is to be executed before or after the triggering statement. BEFORE and AFTER apply to both statement and row triggers.

BEFORE and AFTER triggers fired by DML statements can be defined only on tables, not on views. However, triggers on the on the base table(s) of a view fired if an INSERT, UPDATE, or DELETE statement is issued against the view.

BEFORE and AFTER triggers fired by DDL statements can be defined only on the database or a schema, not on particular tables.

INSTEAD-OF Triggers These triggers provide a transparent way of modifying views that cannot be modified directly through SQL DML statements. These

triggers are called INSTEAD-OF triggers because, unlike other types of triggers, Oracle fires the trigger instead of executing the triggering statement.

Guidelines for Designing Triggers

- Design triggers to:
  - Perform related actions
  - Centralize global operations

- Do not design triggers :
  - Where functionality is already built into the Oracle server
  - That duplicates other triggers

- Create stored procedures and invoke them in a trigger, if the PL/SQL code is very length.

- The excessive use of triggers can result in complex interdependencies, which may be difficult to maintain in large applications.

How triggers are Used?

Triggers can supplement the standard capabilities of Oracle to provide a highly customized database management system.

For example,

- A trigger can restrict DML operations against a table to those issued during regular business hours.
- A trigger could also restrict DML operations to occur only at certain times during weekdays.

Other uses of triggers are to

- Prevent invalid transaction
- Enforce complex security authorizations
- Enforce complex business rules
- Gather statistics on table access
- Modify table data when DML statements are issued against views

Syntax for Table Level Trigger

```
CREATE OR REPLCE TRIGGER <triggername>
BEFORE | AFTER INSERT OR UPDATE OR DELETE ON  <table>
[DECLARE]

BEGIN

     ST1;
    ST2;

[EXCEPTION]

END;
```

Syntax for ROW Level Trigger

```
CREATE OR REPLCE TRIGGER <triggername>
BEFORE | AFTER INSERT OR UPDATE OR DELETE ON  <table>
FOR EACH ROW
[DECLARE]

BEGIN

     ST1;
    ST2;

[EXCEPTION]

END;
```

Example to create Table level or Statement triggers

```
CREATE OR REPLACE TRIGGER secure_emp
 BEFORE INSERT ON emp
BEGIN
   IF TO_CHAR(SYSDATE,'DY') IN('SAT','SUN')  OR
     TO_CHAR(SYSDATE,'HH24:MI') NOT BETWEEN '08:00' AND '18:00' THEN
      RAISE_APPLICATION_ERROR(-20001,'You may Insert into EMP table only
                  during business hours..');
   END IF;
END;
/
```

Note:- RAISE_APPLICATION_ERROR is a server-side built-in procedure that returns an error to the user and causes the PL/SQL block to fail.

When a database trigger fails, the Oracle server automatically rolls the triggering statement back.

Testing secure_emp

```
INSERT INTO EMP (empno, ename) VALUES (101,'ravi');

ERROR at line 1
ORA-20001 You may insert into EMP table only during business hours
ORA-06512  at PLSQL SECURE_emp, LINE 4
Ora-04088 error during execution of trigger "PLSQL SECURE_EMP"
```

Note : The row might be inserted if you are in a different time zone from the database server. The trigger fires even if your system clock is within these business hours.

Combining Triggering Events

You can combine several triggering events into one by taking advantage of the special conditional predicates INSERTING, UPDATING and DELETEING within the trigger body.

Example

Create one trigger to restrict all the data manipulation events on the EMP table to certain business hours, Monday through Friday.

```
CREATE OR REPLACE TRIGGER secure_emp
 BEFORE INSERT OR UPDATE OR DELETE  ON emp
BEGIN

   IF TO_CHAR(SYSDATE,'DY') IN('SAT','SUN')  OR
    TO_CHAR(SYSDATE,'HH24:MI') NOT BETWEEN '08:00' AND '18:00' THEN
       IF INSERTING THEN
         RAISE_APPLICATION_ERROR(-20001,'You may Insert into EMP table
              only   during business hours..');
       ELSIF UPDATING THEN

         RAISE_APPLICATION_ERROR(-20001,'You may UPDATE  EMP table
              only   during business hours..');

       ELSIF DELETING THEN

         RAISE_APPLICATION_ERROR(-20001,'You may DELETE from  EMP
            table only   during business hours..');
       END IF;
   END IF;
END;
/
```

Creating ROW Trigger

You can create a BEFORE row trigger in order to prevent the triggering operation from succeeding if a certain condition is violated.

Create a trigger to allow only certain employees to be able to earn a salary of more than 15,000.

```
CREATE OR REPLACE TRIGGER restrict_salary
  BEFORE INSERT OR UPDATE OF  sal ON emp
  FOR EACH ROW
BEGIN
    IF (:NEW.job  NOT IN ('CLERK', 'SALESMAN')  AND :NEW sal > 15000
          THEN
        RAISE_APPLICATION_ERROR(-20002,'Employees cannot earn this
                    amount');
END IF:
END;
/
```

Using OLD and NEW Qualifiers

Within a ROW trigger, reference the value of a column before and after the data change by prefixing it with the OLD and NEW qualifiers.

Data Operation	Old Value	New Value
INSERT	Null	Inserted value
UPDATE	Value before update	Value after update
DELETE	Value before delete	NULL

- The OLD and NEW qualifiers are available only in ROW triggers
- Prefix these qualifiers a colon (:) in every SQL and PL/SQL statement
- There is no colon (:) prefix if the qualifiers are referenced in the WHEN restricting condition

Note :- Row triggers can decrease the performance if you do a lot of updates on larger tables.

Using OLD and NEW Qualifiers

```
CREATE OR REPLACE TRIGGER AUDIT_EMP_VALUES
AFTER INSERT  ON emp
FOR EACH ROW
BEGIN
   INSERT INTO audit_emp_table
(user_name,timestamp,id,old_name,new_name,old_salary,new_salary)
values(user,sysdate, :OLD.empno, :OLD.ename, :NEW.ename, :OLD.sal,
:NEW.sal);
END;
/
```

Exercise

1) Write a trigger to access the table in a specified time
2) Write a trigger to accept only valid updations (Example, fire the trigger if the employee salary is modified with lesser value than his original salary)
3) Write a trigger to accept unique values in to a particular column of a table, when we are placing a new record.
4) Write a trigger to update stock automatically when customer makes a transaction

Itemmaster

Itemno itemname stock

Itemtran

Transaction_no itemno transaction_date transaction_type Quantity

ROSEDAWSON*315
EXPECTO*315
J@NUARY:-)h0ME315